

COMPTE RENDU

BI & Datawarehouse – Intégration de bases de données sur Talend Open
Studio

3e année Cybersécurité – École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersécurité
Année : 2025/2026
Enseignants : M.RIADSOLH
Date : 14 avril 2026

Table des matières

1	Extraction des données à partir des sources de données	2
2	Transformation	2
2.1	Configuration du tMap_1	2
2.2	Mapping de la sortie Ventes_RH	2
2.3	Gestion des rejets	3
3	Chargement au niveau de la table Ventes_RH	4
3.1	Configuration du tDBOutput	4
3.2	Schéma de la table Ventes_RH	4
3.3	Vérification en base	4
4	Agrégation des données de la table Ventes_RH	6
4.1	Architecture du pipeline d'agrégation	6
4.2	tAggregateRow_1	6
4.3	tSortRow_1	7
4.4	Résultats affichés en console	7
5	Conclusion	8

1 Extraction des données à partir des sources de données

Les données sont extraites depuis la base *Miniature2026* (connectée via les Métadonnées Talend, hôte *localhost*, port *3306*, utilisateur *talend*) à travers quatre composants *tDBInput* :

- **orders** : flux principal (*Main*) vers le *tMap_1*. Contient *orderNumber*, *orderDate*, *requiredDate*, *shippedDate*, *status*, *comments*, *customerNumber*.
- **customers** : flux *Lookup*, jointé sur *orders.customerNumber* = *customers.customerNumber*.
- **employees** : flux *Lookup*, jointé sur *customers.salesRepEmployeeNumber* = *employees.employeeNumber*.
- **offices** : flux *Lookup*, jointé sur *employees.officeCode* = *offices.officeCode*.

Les quatre flux sont connectés en entrée du composant *tMap_1*, comme visible sur la vue générale du job (Figure 4).

2 Transformation

2.1 Configuration du tMap_1

Le composant *tMap_1* effectue les jointures entre les quatre tables et construit deux sorties : *Ventes_RH* (flux principal) et *reject* (lignes sans correspondance).

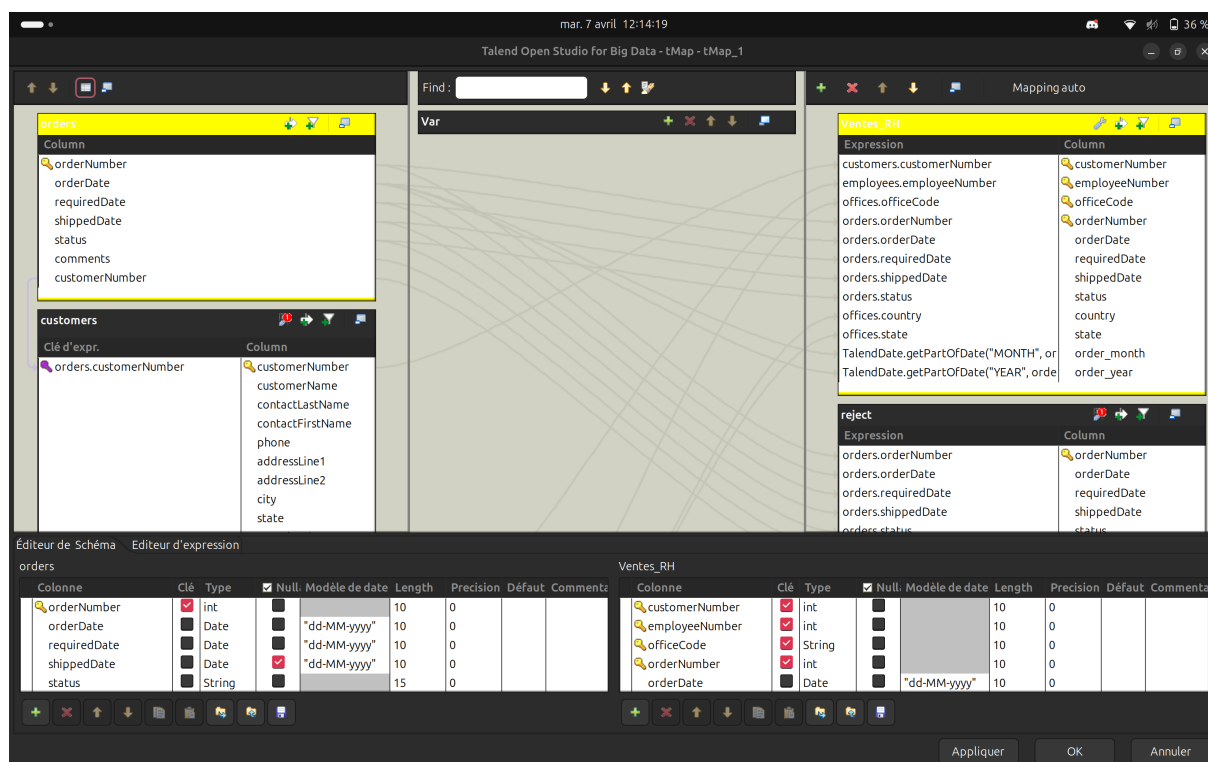


FIGURE 1 – Configuration du *tMap_1* : sources, clés de jointure et sorties

2.2 Mapping de la sortie *Ventes_RH*

Le tableau ci-dessous détaille le mapping défini dans le volet droit du *tMap_1* pour la table de sortie *Ventes_RH* :

Expression source	Colonne de sortie
customers.customerNumber	customerNumber
employees.employeeNumber	employeeNumber
offices.officeCode	officeCode
orders.orderNumber	orderNumber
orders.orderDate	orderDate
orders.requiredDate	requiredDate
orders.shippedDate	shippedDate
orders.status	status
offices.country	country
offices.state	state
TalendDate.getPartOfDate("MONTH", orders.orderDate) + 1	order_month
TalendDate.getPartOfDate("YEAR", orders.orderDate)	order_year

Remarque : `getPartOfDate("MONTH", ...)` retourne 0 pour janvier et 11 pour décembre. Le +1 corrige ce décalage pour obtenir des valeurs de 1 à 12.

2.3 Gestion des rejets

Une seconde sortie **reject** est configurée dans le **tMap_1** avec l'option *Catch inner join reject* activée (**true**). Elle capture les lignes de la table **orders** qui n'ont pas de correspondance dans les tables de lookup. Ces lignes sont écrites dans un fichier CSV via le composant **tFileOutputDelimited_1**.

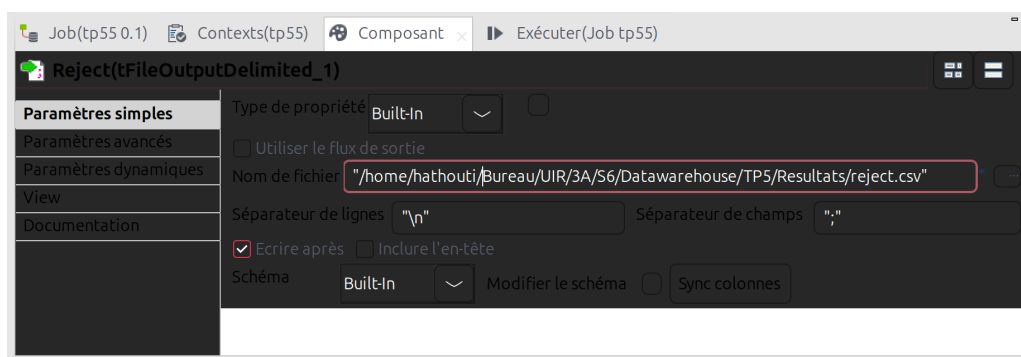


FIGURE 2 – Configuration du composant Reject (tFileOutputDelimited.1)

Le fichier de sortie est enregistré dans :

`/home/hathouti/Bureau/UIR/3A/S6/Datawarehouse/TP5/Resultats/reject.csv`

avec un séparateur de champs ; et l'option *Écrire après* activée. Lors de l'exécution, **0 ligne** a été rejetée, ce qui confirme l'intégrité référentielle entre les tables.

3 Chargement au niveau de la table Ventes_RH

3.1 Configuration du tDBOutput

Le composant tDBOutput_1 (renommé Ventes_RH) charge les données transformées dans la table Ventes_RH de la base Miniature2026. Ses paramètres sont les suivants :

- Hôte / Port : localhost / 3306
- Base de données : Miniature2026
- Utilisateur : talend
- Table : Ventes_RH
- Action sur la table : Supprimer la table si elle existe et la créer
- Action sur les données : Insert

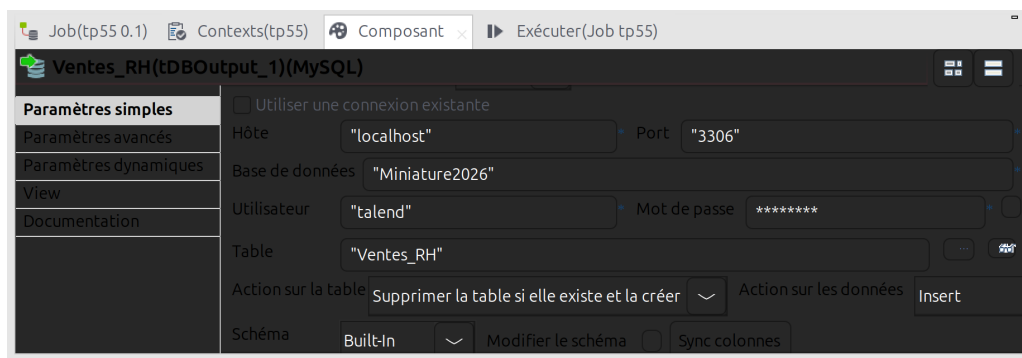


FIGURE 3 – Paramètres de connexion du composant tDBOutput_1 (Ventes_RH)

3.2 Schéma de la table Ventes_RH

Colonne	Type Talend	Type MySQL
customerNumber	int	INT
employeeNumber	int	INT
officeCode	String	VARCHAR(10)
orderNumber	int	INT
orderDate	Date	DATE
requiredDate	Date	DATE
shippedDate	Date	DATE
status	String	VARCHAR(15)
country	String	VARCHAR(50)
state	String	VARCHAR(50)
order_month	int	INT
order_year	int	INT

3.3 Vérification en base

Après exécution du job, la table Ventes_RH est correctement peuplée. La requête de vérification suivante confirme le résultat :

```
1 USE Miniature2026;  
2 SELECT * FROM Ventes_RH;
```

```

Starting job tp55 at 12:15 07/04/2026.
[statistics] connecting to socket on port 4015
[statistics] connected
.---...--+-----+-----+---...--+-----+-----+-----+-----+
|
|                                     tLogRow_1
|                                     |
|=---...--+-----+-----+---...--+-----+-----+-----+-----+
| customerNumber | employeeNumber | officeCode | orderNumber | orderDate | ... |
|   order_month | order_year |
|=---...--+-----+-----+---...--+-----+-----+-----+-----+
|0              |1504          |           |2            |           |...|1
|              |2003          |           |             |           |...|1
|0              |1216          |           |1            |           |...|1
|              |2003          |           |             |           |...|1
|0              |1370          |           |1            |           |...|1
|              |2003          |           |             |           |...|1
|0              |1286          |           |1            |           |...|1
|              |2003          |           |             |           |...|1
|...           |...           |           |...          |           |...|...
|              |...           |           |             |           |...|12
|0              |1216          |           |1            |           |...|12
|              |2004          |           |             |           |...|12
|0              |1165          |           |1            |           |...|12
|              |2004          |           |             |           |...|12
|0              |1611          |           |1            |           |...|12
|              |2004          |           |             |           |...|12
|0              |1286          |           |1            |           |...|12
|              |2004          |           |             |           |...|12
'|---...--+-----+-----+---...--+-----+-----+-----+-----+

[statistics] disconnected
Job tp55 ended at 12:15 07/04/2026. [exit code = 0]

```

4 Agrégation des données de la table Ventes_RH

4.1 Architecture du pipeline d'agrégation

En aval du composant Ventes_RH (tDBOutput_1), un pipeline d'agrégation et de tri est branché pour suivre l'activité des employés :

Ventes_RH $\xrightarrow{\text{row1}}$ tAggregateRow_1 $\xrightarrow{\text{row2}}$ tSortRow_1 $\xrightarrow{\text{row3}}$ tLogRow_1

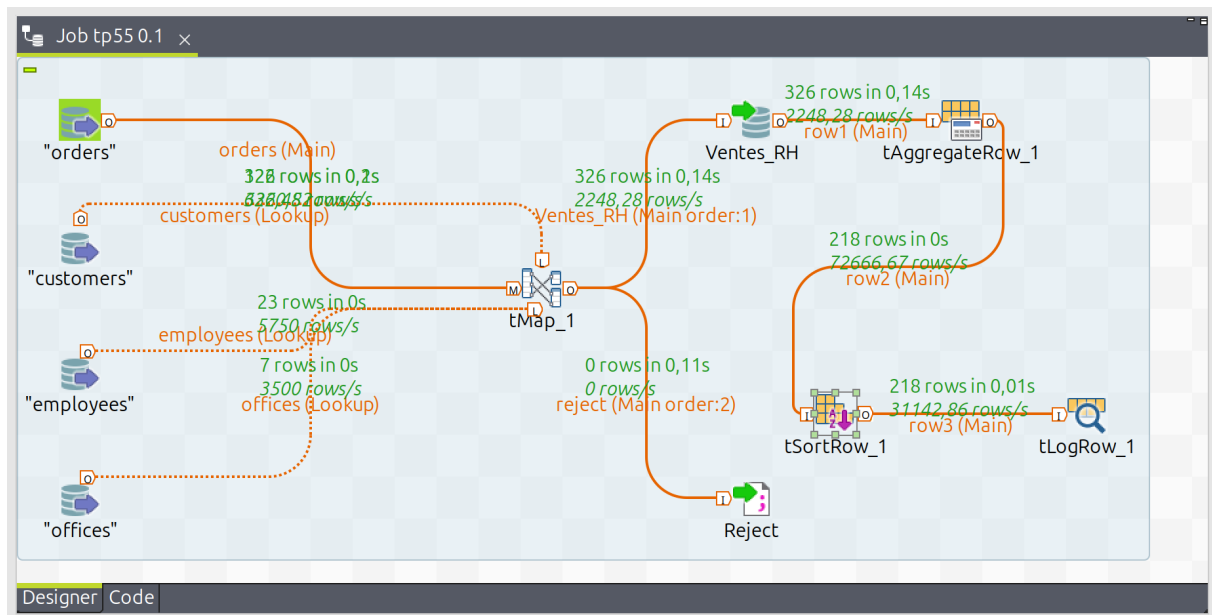


FIGURE 4 – Vue complète du Job tp55 avec pipeline d'agrégation

4.2 tAggregateRow_1

Le composant tAggregateRow_1 agrège le nombre de commandes traitées par employé, par mois et par année. Sa configuration est la suivante :

- **Group by** : employeeNumber, order_month, order_year
- **Opération** : COUNT(orderNumber) → colonne de sortie orderNumber

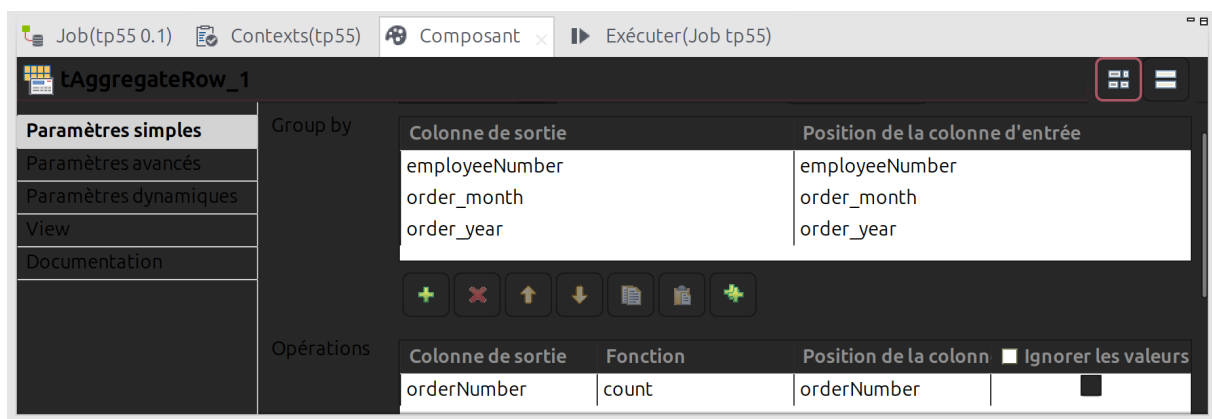


FIGURE 5 – Configuration du tAggregateRow_1

4.3 tSortRow_1

Le composant tSortRow_1 trie les résultats agrégés selon les critères suivants :

Colonne	Type de tri	Ordre
order_month	num	asc
order_year	num	asc
orderNumber	num	desc

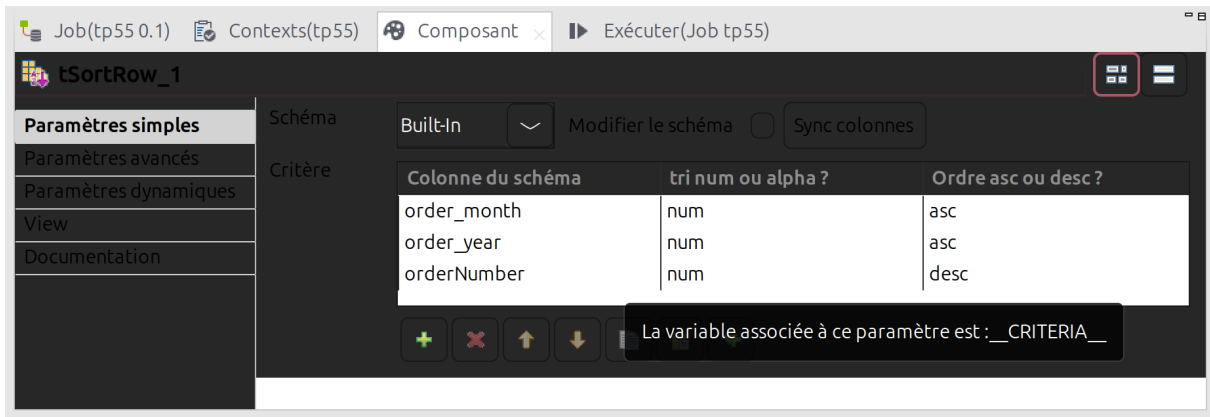


FIGURE 6 – Configuration du tSortRow_1

4.4 Résultats affichés en console

Les résultats triés sont affichés par tLogRow_1 en mode tableau. Chaque ligne représente le nombre de commandes traitées par un employé pour un mois et une année donnés. Le job s'est terminé avec le code de sortie 0, confirmant l'exécution sans erreur.

```
Starting job tp55 at 12:15 07/04/2026.
[statistics] connecting to socket on port 4015
[statistics] connected
.-----..
|                                     tLogRow_1                                     |
|=====..
|customerNumber|employeeNumber|officeCode|...|order_month|order_year|
|=====..
|0  |1504 |null  |2  |null  |null  |null  |null  |1  |2003 |
|0  |1216 |null  |1  |null  |null  |null  |null  |1  |2003 |
|0  |1370 |null  |1  |null  |null  |null  |null  |1  |2003 |
|0  |1286 |null  |1  |null  |null  |null  |null  |1  |2003 |
...
[statistics] disconnected
Job tp55 ended at 12:15 07/04/2026. [exit code = 0]
```

5 Conclusion

Ce TP a permis de mettre en œuvre un pipeline ETL complet sous Talend Open Studio pour le suivi de l'activité des employés, en utilisant la base `Miniature2026` comme source et destination. Les étapes couvertes sont :

- **Extraction** : lecture de quatre tables MySQL (`orders`, `customers`, `employees`, `offices`) via des composants `tDBInput`, avec récupération des schémas depuis les Métadonnées Talend.
- **Transformation** : jointures multi-sources dans le `tMap_1` avec les clés de jointure appropriées entre les tables, calcul des colonnes dérivées `order_month` et `order_year` via `TalendDate.getPartOfDate`, et gestion des rejets de jointure vers un fichier CSV (0 rejet constaté).
- **Chargement** : insertion dans la table `Ventes_RH` via `tDBOutput` avec recreation automatique de la table. 326 lignes ont été chargées avec succès, vérifiées par requête SQL directe.
- **Agrégation** : utilisation de `tAggregateRow` (COUNT des commandes groupé par employé, mois et année), `tSortRow` (tri croissant par mois et année) et `tLogRow` pour visualiser le nombre de commandes traitées par chaque employé dans le temps.